

DoomDraft: Collaborative Text Editing on Simulated Paxos

Aengus McGuinness
CS 2620 – Distributed Systems Final Project

May 12, 2026

Abstract

DoomDraft is a collaborative text editor backed by a three-replica Paxos cluster running in the Cotamer simulator. The system stores each edit as an operation in a replicated PancyDB log. Paxos provides the total order of committed operations, while Operational Transformation (OT) lets browser clients edit optimistically and later reconcile against that committed order. The final project includes a simulation harness with failure schedules, unit tests for the editing layer, and a live HTTP/SSE demo where two browser tabs can edit the same document while replicas fail. The main result is a working application that connects consensus, application-level conflict handling, and a visible browser experience.

1 Introduction

Collaborative editing is a useful stress test for distributed systems because it combines replicated agreement with an interactive user experience. A replicated backend must decide one authoritative order of edits despite failures. At the same time, an editor cannot block on consensus for every keystroke without feeling broken, so each client also needs a local view that may briefly differ from the committed history.

DoomDraft separates these two problems. Paxos is responsible for the durable, fault-tolerant ordering of edits. The collaboration layer is responsible for making each client’s pending local edits consistent with newly committed remote edits. The resulting system is intentionally more than a key/value store demo: it has a real browser editor, document creation, live updates through Server-Sent Events (SSE), cursor reporting, replica failure controls, and automated convergence tests.

The project goal was not to reproduce the full complexity of Google Docs. Instead, DoomDraft aims to demonstrate a coherent path from consensus to application semantics. The central question is: if Paxos gives us a single operation log, can a small editor use that log to keep multiple optimistic clients synchronized?

2 Related Work

DoomDraft sits between two well-known families of collaborative editing systems. Operational Transformation, used in systems such as Jupiter, represents edits as operations and transforms concurrent operations so that clients converge after applying them in different local contexts. Conflict-free Replicated Data Types take a different approach by designing data structures whose operations commute by construction. DoomDraft uses OT because the project already has an ordered replicated log, and OT maps naturally onto character-level insert and delete operations.

The replication layer follows the Paxos style of state-machine replication: clients submit commands, replicas agree on an order, and deterministic replay reconstructs state. In DoomDraft, the commands are not arbitrary database writes from the user’s perspective, but rather are document

operations with editor-specific meaning. Paxos can say which edit happened first, but it does not know how to shift a pending insertion after someone else inserted text before it. OT supplies that application logic.

The project also borrows a common idea from log-structured systems in that it makes the history authoritative and derives the present state from it. This is less efficient than maintaining only the current document string, but it is a good fit for a final project because the log is auditable. If an edit appears wrong in the browser, the operation log shows whether the problem is in agreement, replay, or client-side transformation. This made the design easier to reason about.

3 Design

The document is represented as an append-only operation log rather than a mutable string. Each edit is stored in PancyDB under a key containing the document id, client id, and client sequence number:

$$\text{doc}/\langle \text{doc_id} \rangle / \text{op} / \langle \text{client_id} \rangle / \langle \text{client_seq} \rangle \mapsto \text{I pos text or D pos len.}$$

To reconstruct the current text, the server scans the document’s operation keys, sorts entries by PancyDB version, and replays them from the empty string. Since PancyDB versions reflect Paxos commit order, all live replicas should reconstruct the same document if consensus is working correctly.

The OT layer supports two operation types: insertions and deletions. Its main function transforms operation a over operation b , meaning it rewrites a ’s position and length for the world where b has already been applied. The implementation covers insert/insert, insert/delete, delete/insert, and the usual overlap cases for delete/delete. The core correctness property is the diamond condition:

$$\text{apply}(\text{apply}(\text{doc}, a), \text{transform}(b, a)) = \text{apply}(\text{apply}(\text{doc}, b), \text{transform}(a, b)).$$

Same-position concurrent inserts require a tie-breaker; in the integrated system, operation identity gives that order, while the standalone randomized OT test skips pure same-position insertion collisions.

Each editor client, both in simulation and in the browser, maintains a committed server text plus a queue of local operations that have been applied optimistically but not yet acknowledged. When a remote committed operation arrives, the client applies it to the committed base and transforms every pending local operation over it. When the client’s own operation is acknowledged, that operation is removed from the pending queue. This is the key design pattern: clients can be responsive locally while still converging toward the Paxos order.

This design also gives a useful separation between durable document state and transient interface state. Document operations, document creation, and the document registry are written through the replicated backend because losing or reordering them would change the user’s work. Cursor positions are different however. While they make the demo feel collaborative, actually they are only hints about another tab’s current selection. DoomDraft currently stores cursors through the same path for simplicity.

One subtle consequence of the operation-log design is that the server does not need to maintain a complex mutable document object inside Paxos. The replicated object is just PancyDB. The editor layer can be rebuilt by replaying committed operations, which made debugging much easier. For example, if two clients disagreed, I could ask whether the replicas disagreed about the operation log or whether a browser had transformed its local pending queue incorrectly. That distinction narrowed several bugs from “Paxos is broken” to “the client has misclassified an SSE event.”

The design is also intentionally conservative about what it promises during failures. If a client has applied an edit optimistically but the edit has not committed, that local view is provisional.

Once the edit appears in the Paxos-ordered log, it becomes durable document history. The browser may temporarily be ahead of the server, but the server’s ordered operation stream is the source of truth.

4 Implementation

The C++ simulation model extends the existing PancyDB/Paxos client. Simulated editors randomly generate insertions and deletions, submit them through Paxos, poll for committed operations, and check convergence by reconstructing the document from each live replica. This gave us a fast test surface for the distributed-state property before adding any browser complexity.

The live demo is served by `pt-collab-server`. It embeds three Paxos replicas and exposes a small HTTP interface over Cotamer TCP primitives. The most important endpoints load document text, submit edits, stream SSE updates, manage the document list, and fail a selected replica for the demo. The browser client is plain HTML and JavaScript, with one document operation in flight at a time.

The SSE stream is the bridge between the replicated log and the user’s visible editor. A browser tab first loads a document snapshot, then listens for later operations. For every incoming event, the tab decides whether the operation is stale, its own acknowledged operation, or a remote operation. Only the third case changes the local pending queue by transformation. This classification step is small in code but central to correctness, because a tab that treats its own edit as remote can duplicate text, while a tab that treats another client’s edit as its own acknowledgment can drop an uncommitted local operation.

The document-list feature uses the same philosophy. Creating a new document writes metadata into the replicated registry, and another tab can refresh the list and open that document. This is less algorithmically interesting than editing, but it matters for the demo as an application. The browser is not hard-coded to one document id, and the replicated backend is used for both content and basic application metadata.

Several interesting implementation issues came from crossing the boundary between a simulated Paxos client and real browser requests. First, the server cache could observe a committed operation from polling PancyDB before the original HTTP request resumed. If the cache did not know the browser’s string client id yet, SSE could mislabel the operation and the originating browser would treat its own edit as remote. The fix was to remember the mapping from hashed client ids to browser client ids as soon as a POST arrives.

Second, acknowledgments have to match both client id and sequence number. Every browser tab begins with sequence number zero, so using only the sequence number can cause one tab to accidentally acknowledge another tab’s operation. Third, the HTTP bridge multiplexes many browser requests through one simulated Paxos client id. The original response queue code could erase responses for other pending serial numbers, causing a committed operation to appear in SSE while its original POST timed out. Preserving nonmatching responses for their correct waiter fixed that liveness bug.

A final implementation choice was to keep the browser editor relatively small and explicit. Rather than introduce a rich-text editor framework, the demo uses a plain editable surface and represents changes as character operations. This kept the focus on the distributed protocol and synchronization logic. It also made the browser’s state inspectable: the important variables are the committed server text, pending local operations, the last seen version, and the active SSE connection. For this project, debuggability was more valuable than feature-rich editing.

5 Evaluation

The evaluation has three layers. First, `test-doc-ops` checks the OT transform cases, serialization, sequence transformation, and randomized diamond trials. Second, `test-doc-state` checks reconstruction from the replicated operation log, including version ordering across clients. Third, `pt-collab -test` runs fixed-seed collaboration simulations under unstable membership, failover, and recovery schedules. The latest full run passed these tests.

Layer	Property	Test
OT	Local transform correctness	<code>test-doc-ops</code>
State replay	Text from Paxos-ordered log	<code>test-doc-state</code>
Replication	Live replicas converge	<code>pt-collab -test</code>
Browser path	Two-tab manual demo	HTTP/SSE logs

Table 1: Main validation layers.

The simulation tests primarily evaluate safety. Once operations commit, live replicas reconstruct the same text. The browser demo evaluates a different end-to-end property: edits move from DOM events to HTTP POSTs, through Paxos, into PancyDB, back out through SSE, and into another tab’s local editing state. Aggressive logging was useful here because some failures were not consensus failures. In particular, an operation could commit successfully while the original HTTP request timed out because its response had been consumed by the wrong waiter. This distinction between safety and liveness became one of the main lessons of the project.

Manual testing also confirmed the intended failure behavior. With a Paxos quorum available, the editor can continue accepting document edits after a replica is failed from the UI. If too much of the cluster is unavailable, the browser reports a send failure instead of silently inventing a divergent local history.

The evaluation has two important limitations. First, it does not measure throughput or latency as a systems benchmark. Second, the browser testing is partly manual because the most important observable behavior is two live editing surfaces staying synchronized while a user types. The automated tests compensate by covering the pure OT and replicated-state pieces with deterministic checks, while the browser logs cover the integration path that is hardest to model in a small unit test.

Even with those limitations, the tests found the right class of problems. The fixed-seed simulations caught mistakes where replicas would reconstruct different final documents. The browser logs caught bugs where Paxos had actually committed the right operation, but the web client or HTTP bridge handled the result incorrectly. That split is reassuring: it suggests the evaluation was not merely testing one happy path, but exercising both the distributed backend and the application boundary built on top of it.

The most useful manual scenario used two browser windows connected to the same document. One window typed text while the other watched the SSE stream and applied remote operations; then a replica was failed and more text was entered. This scenario is small, but it exercises nearly every layer: DOM diffing, HTTP submission, Paxos commit, database replay, SSE fanout, pending-operation acknowledgment, and remote transformation. It also makes failures visible immediately. A one-character duplication or dropped acknowledgment shows up as two tabs disagreeing about the text.

I did not treat the live demo as a replacement for automated tests. Instead, it served as an integration test for assumptions that the unit tests cannot easily express. For example, the OT unit tests can prove that a pair of transformed operations commute, but they cannot prove that an SSE

event carries the right browser client id. Similarly, the Paxos simulation can prove that replicas converge, but not that a real HTTP waiter receives the response meant for its serial number. The final evaluation therefore combines small formal-ish checks with end-to-end observation.

6 Conclusion

DoomDraft demonstrates a complete collaborative application on top of simulated Paxos. The most important design choice was representing documents as Paxos-ordered edit logs rather than replicated strings. That made the backend simple and gave the collaboration layer a clear job: transform optimistic local operations around the committed order.

The system still has obvious future work. Cursor updates are currently replicated even though they are ephemeral UI state even though it would make more sense to send them outside Paxos. The document list could update automatically across tabs instead of relying on manual refresh or selection. The HTTP/SSE server could also move from polling toward event-driven fanout. Even with those limitations, DoomDraft satisfies the central goal: it shows how consensus, operation transformation, failure handling, and a user-facing editor can fit together in one small distributed system.

References

- [1] C. A. Ellis and S. J. Gibbs. “Concurrency Control in Groupware Systems.” *ACM SIGMOD*, 1989.
- [2] D. A. Nichols, P. Curtis, M. Dixon, and J. Lamping. “High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System.” *UIST*, 1995.
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. “Conflict-free Replicated Data Types.” *SSS*, 2011.
- [4] L. Lamport. “Paxos Made Simple.” *ACM SIGACT News*, 2001.
- [5] I. Hickson. “Server-Sent Events.” W3C Recommendation, 2015.